Final Report: Process Reward-Guided ToT -MCTS for Mathematical Reasoning

Bhargav Borah (337001005) **Yufeng Yang (334004582)**

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities across diverse natural language tasks, yet they often struggle with multi-step mathematical reasoning—a domain requiring systematic exploration of solution pathways. This study investigates an approach that integrates Tree-of-Thought reasoning based on Monte Carlo Tree Search (MCTS) with process-based rewards to enhance mathematical problem-solving without the computational overhead of reinforcement learning from human feedback (RLHF). As a result, our proposed methods [1], performs no gradient updates on the LLM during inference, operating with a single frozen $1.5B$ parameter model (4-bit quantized) that consumes less than $1GB$ of VRAM. Experimental results on a subset of GSM8K demonstrate that self-critic-based process rewards enhance solution accuracy over the terminal-reward-only baseline. On the other hand, we find log-likelihood-based process rewards discourage exploration of MCTS , limiting the performance of the reasoning framework.

1 Introduction

Recent advances in large-language models (LLMs) have enabled impressive performance on complex reasoning tasks [Moshkov et al., 2025, Chen et al., 2025, Yan et al., 2025, Zhang and Graf, 2025]. However, mathematical problem-solving remains challenging, as it requires reliable multi-step logical deduction, high arithmetic precision, and systematic exploration of solution strategies [Boye and Moell, 2025]. While prompting techniques such as Chain-of-Thought (CoT) [Wei et al., 2022, Wang et al., 2022] have shown promise for eliciting step-by-step reasoning, and search-based methods such as Tree-of-Thoughts (ToT) [Yao et al., 2023] have enabled exploration of multiple reasoning paths, these approaches typically rely on terminal feedback upon task completion, thereby missing opportunities to correct or guide intermediate reasoning steps [Mukherjee et al., 2025, Hou et al., 2025].

To enhance reasoning performance, a dominant approach has been Reinforcement Learning with Verifiable Rewards (RLVR) [Lambert et al., 2024, Guo et al., 2025], which fine-tunes model parameters using verifiable correctness signals through policy optimization methods such as PPO [Zheng et al., 2023], GRPO [Shao et al., 2024], or closed-form reward modeling techniques like DPO [Rafailov et al., 2024]. While RLVR consistently improves model reasoning capabilities, it incurs prohibitive computational costs due to large-scale roll-outs, reward computation, and multi-stage policy optimization. As model sizes scale beyond billions of parameters, the expense of repeated gradient-based post-training becomes impractical for many applications.

Tree-search algorithms present an alternative paradigm that preserves the benefits of structured exploration without requiring gradient updates. The success of MCTS has been demonstrated in AlphaGo and AlphaZero [Silver et al., 2016, 2017a], which achieved superhuman performance in board games (Go, chess, and shogi) by combining neural networks, RL techniques, and MCTS. These

[1] Github Codebase

board game domains require long-term planning and search in vast state spaces—characteristics that are also essential for solving mathematical problems.

However, applying MCTS to LLM reasoning presents distinct challenges compared to game-playing domains. First, the action space in mathematical reasoning is substantially larger and less structured than in board games, where legal moves are well-defined and constrained. Second, intermediate reasoning steps in mathematical problem-solving can often be verified for correctness independent of the final answer, unlike in games where the value of a move is only revealed through complete roll-outs to terminal states. Third, the branching factor in natural language reasoning is orders of magnitude higher than in board games, making exploration computationally prohibitive. To address these challenges, Process Reward Models (PRMs) [Setlur et al., 2024, Lightman et al., 2023, Setlur et al., 2024] have been introduced to provide step-wise feedback during the reasoning process, offering more granular guidance compared to terminal-only rewards [2]

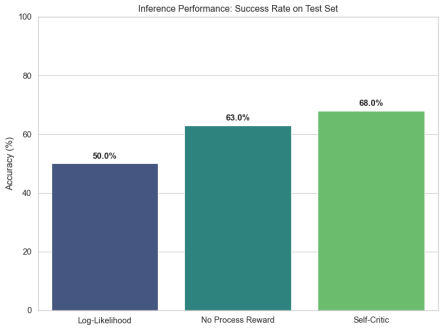


Figure 1: Inference Performance: Success Rate on Test Set

Inspired by Hao et al. [2023], Zhou et al. [2024], Qi et al. [2024], we hypothesize that sophisticated tree search methods, particularly MCTS [Coulom, 2006] combined with ToT-style structured exploration and guided by process rewards, can provide a viable approach for solving mathematical reasoning problems that bridges the gap between linear prompting methods and expensive gradient-based training. To validate this, we organize our final project as follows. In Related Work Section 2, we review prior works on utilizing MCTS and PRMs for LLM reasoning benchmarks. In Methods Section 3, we described the MCTS structure, reward design, and lightweight policy network, which is trained via experience replay on collected search trajectories and the frozen backbone of our fine-tuned LLM. In Experiment Section 4, we employ a compact fine-tuned version of DeepSeek-R1-Distill-Qwen-$1.5B$, demonstrating the efficacy of this method with smaller models. We also test two types of intermediate reward signals to guide MCTS exploration at each reasoning step: (1) symbolic verification that validates mathematical correctness through automated equation checking, and (2) log-likelihood scores that measure reasoning fluency. Finally, we summarize our empirical observations and limitations for guiding future works.

2 Related Works

MCTS Monte Carlo Tree Search (MCTS) was first introduced by Coulom [2006], where they proposed incrementally constructing a search tree through stochastic simulations. Later, Kocsis and Szepesvári [2006] developed the influential Upper-Confidence-bounds-applied to-Trees (UCT) algorithm, which grounded MCTS in multi-armed bandit theory and established a principled exploration& exploitation mechanism. MCTS gradually became the state-of-the-art in computer Go and laid the foundation for a new generation of learning-based game-playing systems. AlphaGo [Silver et al., 2016], which combined deep policy, value networks with MCTS to achieve superhuman performance. Its successor AlphaZero [Silver et al., 2017a] further demonstrated that MCTS-guided self-play can bootstrap strong strategies from scratch, leading to the unified framework that generalized this paradigm to Go, Chess, and Shogi.

MCTS in LLM reasoning Research on integrating MCTS into LLM reasoning can be broadly categorized into two streams. The first stream focuses on test-time integration of MCTS algorithms. Hao et al. [2023] replaced Tree-of-Thought (ToT) with a more sophisticated MCTS approach that treats the LLM itself as a world model, enabling self-evaluation of reasoning trajectories under heuristic reward models rather than relying on separate roll-out policies or supervised learning policy networks. However, Zhou et al. [2024] demonstrated that LLMs cannot achieve effective

[2]Though we are inspired by PRMs, we are not using a reward model in our framework. Hence, when we mention process rewards, we specifically mean the intermediate, non-terminal rewards.

self-correction using internal knowledge alone, highlighting the crucial role of external environment feedback. Their method extends the action space to include both environment-permissible actions and language-based reasoning traces, allowing LLMs to incorporate environment feedback during reasoning. More recently, Qi et al. [2024] addressed MCTS reasoning performance in smaller language models (SLMs) by modeling reasoning as a self-play mutual generation-discrimination process: the target SLM generates candidate reasoning trajectories using MCTS while another SLM acts as a discriminator, providing unsupervised feedback based on partial hints. These test-time approaches—whether leveraging environment feedback or utilizing knowledge from auxiliary models—have demonstrated significant improvements in reasoning performance.

The second stream integrates MCTS with post-training algorithms for training data or roll-out trajectory selection. Xie et al. [2025] adopted an AlphaZero-style self-play mechanism [Silver et al., 2017a] in which MCTS iteratively generates preference responses for sampled prompts, and the resultant data is used to fine-tune model parameters through DPO loss optimization [Rafailov et al., 2024]. Tian et al. [2024] proposed a self-improvement framework where MCTS samples option-level roll-out trajectories: given synthetic prompts, the LLM policy model generates actions for several steps until option-level MCTS terminates, after which a critic model assigns scores by combining process and outcome rewards. These scored trajectories are then used to update the LLM policy via REINFORCE [Williams, 1992]. Similar approaches also appear in Zhang et al. [2024], where MCTS collects high-quality reasoning traces while process reward models (PRMs) provide per-step value estimates for training both policy and reward models. Additional innovations include Gao et al. [2024], enhanced reward modeling through contrastive decoding and achieved an average 51.9% speed improvement per node using speculative decoding. Xu et al. [2025] applied MCTS reasoning to construct high-quality datasets for code-generation fine-tuning tasks.

PRMs in LLM PRMs have become important for improving LLM reasoning by providing step-level supervision and additional guidance rather than relying solely on outcome rewards. They have been used in many regimes including test-time guidance [Ma et al., 2023, Zhao et al., 2025, Zou et al., 2025], steering chain-of-thought generation [Khalifa et al., 2025], and learning value functions within MCTS frameworks [Zhang et al., 2024]. Specifically, Zhang et al. [2024] integrates process reward guidance with MCTS for collecting high-quality reasoning traces and step-values, which are then used to train policy and reward models. By performing a sufficient number of roll-outs, this "labeling" process effectively filters out the subset of samples with the highest quality. Similarly, Park et al. [2025] employs PRMs to evaluate reasoning steps during MCTS over ensembles of LLMs, improving reliability on mathematical reasoning tasks. In summary, these works highlight PRMs' potential as natural complements to MCTS, enabling more reliable search, better trajectory selection, and improved decision-making across various reasoning tasks.

Mathematical Reasoning Datasets and Benchmarks Mathematical reasoning has emerged as a critical testbed for evaluating LLM capabilities, prompting the development of several benchmark datasets. GSM8K [Cobbe et al., 2021] consists of $8.5K$ grade-school math word problems that require multi-step reasoning, making it a widely adopted benchmark for assessing basic mathematical problem-solving abilities. Though arguably saturated, some of the strongest contenders on this benchmark are open-source models such as Kimi K2 [Team et al., 2025], Llama 3.1 $405B$ Instruct [Team, 2024], Qwen-2.5 $32B$ Instruct [Qwen Team, 2024a], and Qwen-2.5 $72B$ Instruct [Qwen Team, 2024b]. The MATH dataset [Hendrycks et al., 2021] presents a challenging suite of competition-level mathematics problems spanning algebra, geometry, number theory, and other Olympiad-style domains. Frontier reasoning models such as DeepSeek-R1 [Guo et al., 2025], Kimi K2 [Team et al., 2025], and inference frameworks Xolver [Hosain et al., 2025], self-check [Miao et al., 2023] leveraging cognitive tools or self-verification have recently achieved near-saturated performance on MATH-500. MathQA [Amini et al., 2019] provides a large-scale collection of math word problems with multiple-choice answers, emphasizing symbolic operations and algebraic reasoning. Self-checking and program-of-thought frameworks [Miao et al., 2023, Yu et al., 2024] have shown consistent improvements on MathQA over standard chain-of-thought decoding.

3 Methods

In this section, we introduce the proposed formulation in detail. Our approach employs Monte Carlo Tree Search (MCTS) to generate reasoning traces. These data are subsequently used to train the policy network, refining its action-selection probabilities (priors). The improved policy network

then provides enhanced guidance for node expansion in subsequent MCTS searches, establishing a process that iteratively enhances both the search process and the policy network's capability to identify promising reasoning trajectories.

3.1 MCTS Search Phase

For each problem, we initialize the empty reasoning step, decompose the problem state as input and extract the answer as the ground truth. When leveraging MCTS for searching the predicted answer, we initialize the root node with the problem statement and empty reasoning steps; the child nodes represent the intermediate reasoning steps until the final answer is reached. To guide the search, we utilize a policy network π_θ which predicts the prior probability $P(s, a)$ of selecting a specific reasoning step, and we maintain state-action value, $Q(s, a)$, which is the total accumulated reward averaged by visit counts $N(s, a)$. At each iteration, MCTS executes the following four steps.

Selection To select the reasoning path, we use the Upper Confidence Bound for Trees (UCT) variant used in AlphaZero to achieve an exploration and exploitation balance. Specifically, at each reasoning step s, we select the child node a according to

$$a = \arg\max_a \big[Q(s, a) + c_{\text{puct}} \cdot P(s, a) \cdot \frac{\sqrt{\sum_{b \in A(s)} N(s, b)}}{1 + N(s, a)}\big], \tag{1}$$

where c_{puct} is a hyper-parameter constant controlling exploration, $P(s, a)$ is the prior probability predicted by the policy network, and $N(s, a)$ is the node visit count.

Expansion If the selected node is not terminal and the tree depth has not reached the pre-defined maximal depth, we expand the tree. We input the current reasoning step s into the LLM to generate k candidate reasoning steps. These candidates are instantiated as child nodes, and their prior probabilities $P(s, a)$ are initialized using the policy network's predictions.

Simulation and Reward Design After expansion, we perform a simulation (roll-out) from the new node until a terminal state is reached or the maximum depth is exceeded. During the roll-out, we calculate the process reward for each step (e.g., using the LLM's self-critic verification score or normalized log-likelihood). We accumulate these discounted step rewards to form the total path reward. Specifically, the terminal reward is designed such that if the predicted answer equals the ground truth answer, the final reward is set to 10.0; if the predicted answer is incorrect, it is set to -5.0; If the search fails to reach a valid terminal state, a penalty of -5.0 is applied.

Backpropagation After receiving the total discounted reward R from the simulation, we apply backpropagation to update the visit counts and value estimates of all visited nodes along the path through

$$N(s, a) \leftarrow N(s, a) + 1; \quad Q(s, a) \leftarrow \frac{Q(s, a) \cdot (N(s, a) - 1)R}{N(s, a)}. \tag{2}$$

3.2 Policy-Network Training Phase

In order to guide the MCTS towards more favorable reasoning paths during inference, we train a lightweight policy network to evaluate reasoning steps. In Salahi et al. [2024], LLMs were fine-tuned to act as value function for evaluation of states. However, given computational constraints, we delegate the functionality of the value function to process rewards. Importantly, the performance of the reasoning system is highly dependent on tuning related hyper-parameters and the choice of metric used as the process-reward proxy.

Training Data Collection We train on a subset of 100 question-answer pairs from GSM8K, stratified by problem difficulty. This stratification allows the reasoning model to learn progressively, i.e., mastering easier problems before advancing to more difficult ones with longer reasoning sequences (see Appendix B for details).

For each problem, we perform the MCTS search described above and extract training examples (s, π) from every visited non-terminal node in the resulting search tree. Here, s represents the state embedding, and π is the target policy distribution derived from visit counts. Inspired by Silver et al. [2017b], to guide the network toward more robust solution paths and accelerate learning, we apply

temperature sharpening by cubing the visit counts, so that the target probability for action a becomes: $\pi(a|s) \propto N(s,a)^{3.0}$ (see Appendix C for more details on stabilization of the learning of the policy network).

Policy Network Architecture We design the policy network as a lightweight, three-layer Multi-Layer Perceptron (MLP) that acts as a specialized head on top of a frozen-version of the fine-tuned LLM we use for reasoning sequence generation. The network takes the normalized last-token embedding of the reasoning state as input. It consists of two hidden layers (dimensions 256 and 128) with Layer Normalization and ReLU activation, followed by a final linear projection to a scalar logit. These logits are normalized via Softmax to produce the action probability distribution $P(s,a)$.

Policy Network Training To minimize computational overhead, the base LLM remains frozen, serving solely as a feature extractor and candidate generator. We maintain a First-In-First-Out (FIFO) replay buffer (capacity 10,000) to store collected examples and stabilize training.

4 Experiments

We implemented a Tree-of-Thoughts (ToT) reasoning framework guided by Monte Carlo Tree Search (MCTS). The system comprises two distinct components: a frozen *Generator* (a 4-bit quantized LLM) responsible for producing reasoning steps and state embeddings, and a trainable *Policy Network* for generating process rewards. The Policy Network is a lightweight Multi-Layer Perceptron (MLP) that maps normalized hidden state embeddings to scalar action values, guiding the search toward promising reasoning paths. We used two different types of process rewards for guiding the reasoning, self-critic-based process reward, and log-likelihood-based process rewards. We also trained a baseline which doesn't use process reward, relying solely on verifiable terminal rewards for search.

4.1 Self-Critic-based Process Rewards

To overcome the sparsity of terminal rewards in multi-step reasoning, we used a dense "self-crtic" process reward. For every generated step s_t, the system constructs a verification prompt (which essentially asks: "Is the step above mathematically correct?"). Rather than generating text, the system performs a single forward pass to compute the reward R_{process} based on the model's intrinsic confidence

$$R_{\text{process}} = \sigma\big(\text{logit}_{\text{“Yes”}}, \text{logit}_{\text{“No”}}\big) \times \lambda, \tag{3}$$

where σ represents the softmax function over the binary token pair and λ is a scaling factor (0.5), necessary for ensuring the terminal reward is not overwhelmed by intermediate rewards. Such aforementioned design allows the frozen LLM to act as its own verifier, which we believe helpful on penalizing hallucinations or arithmetic errors immediately.

4.2 Normalized Log-Likelihood-based Process Rewards

We are motivated to experiment with a process reward based on the frozen Generator's own confidence. This approach assumes that reasoning steps with higher probability under the pre-trained distribution are more likely to be coherent and logically consistent. Denote $h_t = [t_1, t_2, \ldots, t_n]$ as the full reasoning history up to step t, which comprise both the initial context and all generated reasoning steps. Then, for current step s_t with length L_{step} tokens, the reward is

$$R_{\text{process}} = \Big(\frac{1}{L_{\text{step}}} \sum_{i=1}^{|h_t|} \log P(t_i \mid t_{<i})\Big) \times \lambda. \tag{4}$$

where λ is the process reward scale, and $|h_t|$ denotes the total token count in the history.

4.3 Result analysis

We present obtained test accuracy results utilizing self-critic process rewards, normalized log-likelihood process rewards and their comparisons against no process rewards in Table 1. We observe that the accuracies were consistently higher for the training data. The categorization of short, medium, and long was based on reasoning traces in the ground truth answers in GSM8K [Cobbe et al., 2021], not the actual number of reasoning traces employed by the model. This can be explained by the fact

Phase	Experiment	Accuracy (%)				Ratio		
		Short (0-4)	Medium (5-8)	Long (9+)	Overall	Short (0-4)	Medium (5-8)	Long (9+)
Test	Log-Likelihood	68.42	46.15	26.09	50.00	26/38	18/39	6/23
	No Process Reward	78.95	58.97	43.48	63.00	30/38	23/39	10/23
	Self-Critic	73.68	66.67	60.87	68.00	28/38	26/39	14/23
Train	Log-Likelihood	84.00	62.50	40.00	71.00	42/50	25/40	4/10
	No Process Reward	90.00	70.00	60.00	79.00	45/50	28/40	6/10
	Self-Critic	90.00	75.00	60.00	81.00	45/50	30/40	6/10

Table 1: Accuracies across Experiments

that the base model was fine-tuned on reasoning traces from GSM8K [Cobbe et al., 2021]. The test problems and its reasoning sequences, however, were not seen by the LLM during finetuning.

The reasoning frameworks which used self-critic based process rewards consistently performed the best during inference on test examples. As a comparison, using log-likelihood led to much lower accuracies, being not able to surpass the baseline's performance. We hypothesize that using normalized log-likelihood discouraged reasoning trajectories with longer paths, a characteristic not fit into mathematical reasoning problems. However, we believe further hyperparameter tuning of the discount factor and scaling factor λ might help alleviate the poor performance when using log-likelihood-based process reward to some extent. In addition, We also plot the ablation test result under different choices of process rewards. Figure 2a, 2b, 2c show the loss value and accuracy when training the policy network.

5 Limitations

Our current experiments were only performed on datasets with 100 training and 100 test problems from GSM8K, which limit statistical reliability and generalizability. Broader evaluation datasets, such as MATH [Hendrycks et al., 2021], MathQA [Amini et al., 2019], and more complex problem sets are needed for furthering the claims. Additionally, the results of the experiments are suboptimal and not robust to hyperparameters as suggested by log-likelihood experiment's poor performance (50% accuracy).

It seems that our proposed approach inherits the limitations of underlying LLM, and relies only on self-critic verification rather than trained Process Reward Models. This proxy signal may be poorly calibrated and lacks the granular supervision of human-annotated step-wise correctness labels.

At last, performance degrading on longer chains (73.68% to 60.87% accuracy), reveals difficulties on maintaining coherence when the action space grows exponentially with reasoning depth. This was observed during manual inspection of reasoning traces for problems whose ground truth answers had long reasoning trajectories.

6 Conclusion

In the final project, we investigated the integration of Monte Carlo Tree Search-based Tree-of-Thought model with process rewards to enhance mathematical reasoning in large language models without requiring expensive reinforcement learning from human feedback. Our experiments demonstrated that incorporating self-critic-based process rewards into MCTS-guided reasoning improves solution accuracy over terminal-reward-only baselines, achieving 68% accuracy on held-out test problems compared to 63% for the baseline approach. Notably, our method operates with a single frozen 1.5B parameter model (4-bit quantized, consuming less than 1GB VRAM), making it computationally practical for resource-constrained environments. The results validate our central hypothesis that dense intermediate feedback can potentially be effective on exploration of reasoning traces. And leveraging the model's intrinsic confidence on mathematical correctness during reward design, such as self-critic process reward, was proven particularly effective, especially on problems requiring longer reasoning chains (60.87% accuracy on long problems vs. 43.48% for the baseline). Overall, the final project suggests that step-wise verification helps the search algorithm avoid error accumulation and navigate complex solution spaces more efficiently.

References

Aida Amini, Saadia Gabriel, Shanchuan Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Long and Short Papers*, pages 2357–2367, 2019. doi: 10.18653/v1/N19-1245.

Johan Boye and Birger Moell. Large language models and mathematical reasoning failures, 2025.

Yang Chen, Zhuolin Yang, Zihan Liu, Chankyu Lee, Peng Xu, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Acereason-nemotron: Advancing math and code reasoning through reinforcement learning. *arXiv preprint arXiv:2505.16400*, 2025.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *International conference on computers and games*, pages 72–83. Springer, 2006.

DeepSeek-AI. Deepseek-r1-distill-qwen-1.5b. `https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B`, 2024. Model repository.

Zitian Gao, Boye Niu, Xuzheng He, Haotian Xu, Hongzhang Liu, Aiwei Liu, Xuming Hu, and Lijie Wen. Interpretable contrastive monte carlo tree search reasoning. *arXiv preprint arXiv:2410.01707*, 2024.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and ... Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173, 2023.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *CoRR*, abs/2103.03874, 2021.

Md Tanzib Hosain, Salman Rahman, Md Kishor Morol, and Md Rizwan Parvez. Xolver: Multi-agent reasoning with holistic experience learning just like an olympiad team. *arXiv preprint arXiv:2506.14234*, 2025.

Yujie Hou, Ting Zhang, Mei Wang, Xuetao Ma, and Hu Huang. Smart: Self-generating and self-validating multi-dimensional assessment for llms' mathematical problem solving, 2025.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, et al. Process reward models that think. *arXiv preprint arXiv:2504.16828*, 2025.

Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *Proceedings of the 17th European Conference on Machine Learning (ECML)*, pages 282–293. Springer, 2006.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tülu 3: Pushing frontiers in open language model post-training, 2024.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

Qianli Ma et al. Step-level reward model as the navigators for reasoning. In *Conference on Neural Information Processing Systems (NeurIPS) Workshops*, 2023.

Ning Miao, Yee Whye Teh, and Tom Rainforth. Selfcheck: Using llms to zero-shot check their own step-by-step reasoning. *arXiv preprint arXiv:2308.00436*, 2023.

Ivan Moshkov, Darragh Hanley, Ivan Sorokin, Shubham Toshniwal, Christof Henkel, Benedikt Schifferer, Wei Du, and Igor Gitman. Aimo-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset. *arXiv preprint arXiv:2504.16891*, 2025.

Sagnik Mukherjee, Abhinav Chinta, Takyoung Kim, Tarun Anoop Sharma, and Dilek Hakkani-Tür. Premise-augmented reasoning chains improve error identification in math reasoning with llms. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, pages 45109–45128. PMLR, 2025. URL `https://proceedings.mlr.press/v267/mukherjee25a.html`.

Sungjin Park, Xiao Liu, Yeyun Gong, and Edward Choi. Ensembling large language models with process reward-guided tree search for better complex reasoning. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025. Long Paper.

Zhenting Qi, Mingyuan Ma, Jiahang Xu, Li Lyna Zhang, Fan Yang, and Mao Yang. Mutual reasoning makes smaller llms stronger problem-solvers. *arXiv preprint arXiv:2408.06195*, 2024.

Qwen Team. Qwen2.5: A party of foundation models. `https://qwenlm.github.io/blog/qwen2.5/`, September 2024a.

Qwen Team. Qwen2.5 72b instruct. `https://huggingface.co/Qwen/Qwen2.5-72B-Instruct`, 2024b.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024.

Kamyar Salahi, Pranav Gurusankar, and Sathya Edamadaka. More effectively searching trees of thought for increased reasoning ability in large language models. Stanford CS224N Custom Project Report, 2024. Presented in Stanford CS224N Winter 2024 final-project collection.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. Rewarding progress: Scaling automated process verifiers for llm reasoning. *arXiv preprint arXiv:2410.08146*, 2024.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017a.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017b. doi: 10.1038/nature24270.

Meta Llama Team. Meta-llama 3.1 405b instruct. `https://huggingface.co/meta-llama/Llama-3.1-405B-Instruct`, 2024. Open-weight large language model.

Moonshot AI Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Chen, Hao Ding, and ... Kimi k2: Open agentic intelligence, 2025.

Ye Tian, Baolin Peng, Linfeng Song, Lifeng Jin, Dian Yu, Lei Han, Haitao Mi, and Dong Yu. Toward self-improvement of llms via imagination, searching, and criticizing. *Advances in Neural Information Processing Systems*, 37:52723–52748, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

Yuxi Xie, Anirudh Goyal, Wenyue Zheng, Min-Yen Kan, Timothy P Lillicrap, Kenji Kawaguchi, and Michael Shieh. Monte carlo tree search boosts reasoning via iterative preference learning. *NeurIPS*, 2025.

Bin Xu, Yiguan Lin, Yinghao Li, and Yang Gao. Sra-mcts: Self-driven reasoning augmentation with monte carlo tree search for code generation. *IJCAI*, 2025.

Yibo Yan, Jiamin Su, Jianxiang He, Fangteng Fu, Xu Zheng, Yuanhuiyi Lyu, Kun Wang, Shen Wang, Qingsong Wen, and Xuming Hu. A survey of mathematical reasoning in the era of multimodal large language model: Benchmark, method & challenges. In *Findings of the Association for Computational Linguistics (ACL 2025)*, pages 11798–11827, 2025.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

Tailin Yu et al. Mammoth: Building math generalist models through hybrid program-of-thought training. `https://tiger-ai-lab.github.io/MAmmoTH/`, 2024.

Dan Zhang, Sining Zhoubian, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. Rest-mcts*: Llm self-training via process reward guided tree search. *Advances in Neural Information Processing Systems*, 37:64735–64772, 2024.

Liang Zhang and Edith Graf. Mathematical computation and reasoning errors by large language models. In *Proceedings of the Artificial Intelligence in Measurement and Education Conference (AIME-Con 2025)*, pages 417–424, 2025.

Jian Zhao, Runze Liu, Kaiyan Zhang, Zhimu Zhou, Junqi Gao, Dong Li, et al. Genprm: Scaling test-time compute of process reward models via generative reasoning. *arXiv preprint arXiv:2504.00891*, 2025.

Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Qin Liu, Yuhao Zhou, Limao Xiong, Lu Chen, Zhiheng Xi, Nuo Xu, Wenbin Lai, Minghao Zhu, Cheng Chang, Zhangyue Yin, Rongxiang Weng, Wensen Cheng, Haoran Huang, Tianxiang Sun, Hang Yan, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. Secrets of rlhf in large language models part i: Ppo, 2023.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *International Conference on Machine Learning*, 2024.

Jiaru Zou, Ling Yang, Jingwen Gu, Jiahao Qiu, Ke Shen, Jingrui He, and Mengdi Wang. Reasonflux-prm: Trajectory-aware prms for long chain-of-thought reasoning. *arXiv preprint arXiv:2506.18896*, 2025.

A Finetuning on Reasoning Traces

A.1 Motivation

The base model selected for this study was DeepSeek-R1-Distill-Qwen-1.5B [DeepSeek-AI, 2024]. While capable of general reasoning, preliminary experiments revealed two critical limitations for our Monte Carlo Tree Search (MCTS) framework:

1. **Verbosity and Redundancy:** The base model tended to generate excessively long, conversational reasoning chains that were computationally expensive to expand during search.
2. **Lack of Structured Output:** The MCTS algorithm requires discrete, well-defined reasoning steps to build the search tree. The base model did not natively output reasoning in a step-by-step format compatible with state transitions, nor did it reliably signal the termination of a reasoning chain. Extraction of the verifiable numerical output from such outputs was also an issue.

To address, we fine-tuned the model to generate reasoning in discrete, single-step increments, and simultaneously predicting a termination flag to signal the end of the solution.

A.2 Dataset Construction

We utilized the training split of GSM8K [Cobbe et al., 2021] to obtain data for fine-tuning our model. We decomposed the ground truth solutions into individual steps using newline delimiters. We then transformed these linear solutions into a set of cumulative training examples using a sliding window approach.

For a problem with a solution consisting of steps $[s_1, s_2, \ldots, s_T]$, we generated T distinct training examples. For any given step t, the input provided to the model included the original problem statement and the history of all steps generated prior to t.

A.3 Input-Output Structure

We enforced a structured output format (JSON) to capture the termination signal (`is_final`).

Input: The input prompt was standardized to include the math problem and the accumulated reasoning history.

- **Format:**

```
Problem: {problem_text}
Steps so far:
{s_1}
{s_2}
...
{s_{t-1}}
```

Output: Instead of generating raw text, the model was trained to output a JSON object containing two fields:

1. `next_step`: The text content of the current reasoning step (s_t).
2. `is_final`: A boolean flag indicating if s_t is the final step of the solution.

- **Format:**

```
{
  "next_step": "...",
  "is_final": true/false
}
```

By including the `is_final` flag directly in the generation target, the model learns to recognize the semantic conditions that signify the conclusion of a proof or calculation.

A.4 Implementation Details

The model was fine-tuned using Supervised Fine-Tuning (SFT) on the causal language modeling objective.

- **Base Model:** DeepSeek-R1-Distill-Qwen-1.5B
- **LoRA Configuration:** We applied Low-Rank Adaptation (LoRA)[Hu et al., 2021] with rank $r = 16$ and alpha $\alpha = 16$, targeting all linear projection modules (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`).
- **Hyperparameters:**
  - Learning Rate: 2×10^{-4}
  - Batch Size: 2 (with gradient accumulation steps = 4)
  - Epochs: 3
  - Optimizer: AdamW (8-bit)

This configuration allowed the model to retain its mathematical reasoning capabilities while strictly adhering to the required step-by-step output format. The fine-tuned model, though, more aligned with our requirements, did make mistakes in the output format, particularly in problems requiring long reasoning trajectories.

B Sampling of GSM8K for dataset creation

To create a dataset that effectively evaluates the reasoning capabilities of the model, we performed a hierarchical sampling of the GSM8K dataset based on the complexity of the reasoning required. Due to computational constraints, we experimented on a small subset of GSM8K rather than the full dataset. We hypothesize that the length of the reasoning chain in the ground truth solution serves as a proxy for problem difficulty. Longer reasoning chains typically requires maintaining coherent state transitions across multiple logical dependencies and performing complex operations. We analyzed the distribution of reasoning traces (defined by newline characters in the solution) and categorized the problems into three difficulty bins:

Short: Requires 0 to 4 reasoning traces.

Medium: Requires 5 to 8 reasoning traces.

Long: Requires ≥ 9 reasoning traces.

We constructed the training and testing subsets to test the model's generalizability to longer reasoning paths. The specific composition of the subsets is as follows:

B.1 Training Subset

The training subset consists of 100 samples, heavily weighted towards shorter and medium-length reasoning chains to establish fundamental reasoning patterns:

- **Short:** 50 samples
- **Medium:** 40 samples
- **Long:** 10 samples

B.2 Testing Subset

The testing subset consists of 100 samples. To rigorously test generalization, we increased the proportion of "Long" and "Medium" problems compared to the training set. Notably, the GSM8K test set contained only 23 problems fitting the "Long" category, all of which were included in our subset:

- **Short:** 38 samples
- **Medium:** 39 samples
- **Long:** 23 samples (All available samples from the original test set)

This distribution ensures that the test set is comparatively more challenging than the training set, specifically targeting the model's ability to handle extended reasoning sequences.

C Policy Network Stabilization Techniques

Initial experiments revealed that the policy network failed to converge when trained on raw Monte Carlo Tree Search (MCTS) visit counts and raw embeddings output from the LLM. To address this instability and ensure effective learning in the low-data regime of reasoning traces, we incorporated five specific stabilization techniques. These methods were largely adapted from the AlphaZero[Silver et al., 2017b] training pipeline and modified for the specific constraints of reasoning trace generation.

C.1 Input Embedding Normalization

The policy network receives raw hidden states from the LLM (dimension $d = 1536$) as input. We observed that the magnitude of these embedding vectors varied significantly, resulting in unstable gradients within the policy head. To mitigate this, we applied L_2-normalization to project all state embeddings onto a unit hypersphere before passing them to the Multi-Layer Perceptron (MLP).

C.2 Target Sharpening via Exponentiation

The training target for the policy network is the distribution of visit counts (N) generated by the MCTS. In the early stages of training, visit counts were often near-uniform, providing weak supervision signals. We applied temperature scaling (sharpening) to the visit counts before normalizing them into a probability distribution. We increased the exponent from the standard 1.0 to 3.0 to force the network to focus decisively on the most visited paths.

C.3 Root Exploration Noise

To prevent the policy from collapsing into a deterministic local optimum—where it repeatedly selects the same "safe" reasoning steps—we injected noise into the search tree. We added Dirichlet noise ($\alpha = 1.0, \epsilon = 0.25$) to the prior probabilities of the root node's children. This ensures that the MCTS explores a diverse set of initial reasoning steps during data collection.

C.4 Architecture Simplification and Regularization

Given the limited number of reasoning traces compared to the high dimensionality of the input, the network was prone to overfitting. We applied the following architectural constraints:

- **Dimension Reduction:** The hidden layer dimension was reduced from 512 to 256.
- **Dropout Removal:** Dropout layers were removed, as they introduced excessive variance given the small batch sizes used in online Reinforcement Learning.
- **Weight Decay:** We introduced L2 regularization ($\lambda = 10^{-4}$) in the optimizer to constrain weight magnitudes.

Sampling Strategy Adjustment

We initially employed Prioritized Experience Replay (PER) [Schaul et al., 2016]. However, in the context of reasoning traces, "high error" samples often corresponded to noisy or invalid reasoning chains rather than informative outliers. We reverted to Uniform Sampling from the replay buffer to ensure a stable distribution of training data.

C.5 Increasing Batch Size

We experimented with a number of different batch sizes, and observed the training loss curves of the policy network, before settling on a batch size of 32. In this choice, we faced a tradeoff: using a lower batch size would mean more frequent gradient updates of the policy network. However, the training would then be much noisier.

Incorporating these measures did help in lowering the policy network's training losses from the range of $0.9 - 1.0$ to $0.5 - 0.6$. However, the loss curve does plateau relatively early into the training, in addition to being noisy, as we can observe in Figure 2a, 2b and 2c.

D Training Dynamics, Process Rewards, and Inference Analysis

In this section, we analyze the training stability of the policy network, compare the impact of different process reward signals.

D.1 Training Dynamics and Loss Convergence

We monitored the policy network's training progress over 100 problem instances from the GSM8K subset, where examples progressively got more difficult in terms of the number of reasoning steps required to solve the problem. Fig 2a, 2b and 2c illustrate the rolling accuracy (window size $k = 20$) and the policy network loss (cross-entropy) for three distinct reward configurations.

- **Loss Convergence:** Across all three configurations, the policy network loss drops sharply within the first 20 training steps (from > 1.1 to ≈ 0.6) and stabilizes. This indicates that the network successfully learns to approximate the MCTS visit count distribution. The residual variance in the loss (oscillating between 0.5 and 0.7) is expected in an online RL setting where the target distribution (MCTS results) is non-stationary and evolves as the policy improves. However, there is scope for improvement here, at least in terms of hyperparameter-tuning.
- **Accuracy Fluctuations:** The cumulative accuracy starts high (1.0) and gradually decreases to the $0.6 - 0.7$ range. This trend is attributed to the curriculum effect: the initial batch of problems in the dataset happened to be simpler, while later problems required longer reasoning chains, increasing the probability of cumulative error. Please note that Fig 2 is an example-wise scatter plot of accuracy, where the difficult problems occur progressively as the policy network learns. Because the hardest problems occur towards the end, there is a downward trend in this plot.

D.2 Impact of Process Rewards

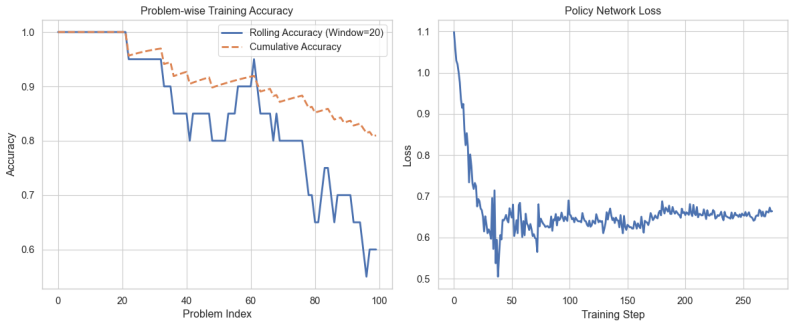
We compared three reward shaping strategies to guide the MCTS:

1. **Self-Critic (Fig.3a):** Uses the model's own verification confidence as a dense reward. This configuration demonstrated the most stable rolling accuracy in the mid-training phase (problems 40-60), suggesting that the self-critic signal helps prune incorrect reasoning branches earlier than sparse terminal rewards.
2. **Log-Likelihood (Fig 3b):** Uses the average token probability of the reasoning step. While the loss convergence is slightly lower (smoother targets), the final accuracy is comparable to the self-critic approach. This suggests that high-probability text generation does not always correlate with mathematical correctness. It is important to add that using log-likelihood-based process rewards led to the poorest performance among the three methods during test inference, scoring a lower accuracy than the no-process-rewards baseline
3. **No Process Reward (Fig 3c):** Relies solely on the terminal reward (± 10). Surprisingly, this baseline remains competitive, especially for problems which can be solved using fewer than reasoning steps, implying that for short reasoning chains, the MCTS lookahead is sufficient to propagate the terminal signal back to the root without dense intermediate rewards.

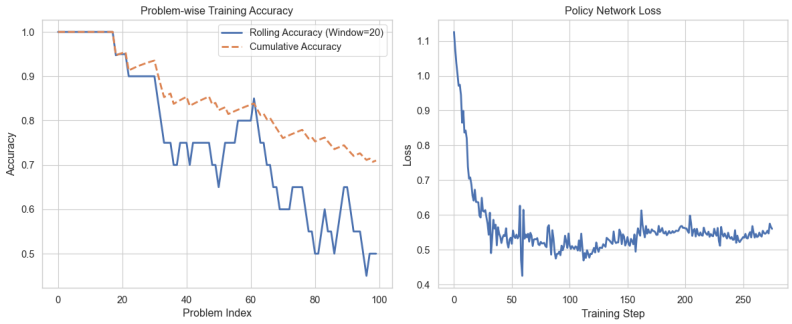
D.3 Inference Analysis

While there is not a clear trend as to whether the reasoning system uses a greater number of reasoning steps when it is right, from left plots in Figs 3a,3b, 3c, we can infer that for problems it eventually answers incorrectly, the reasoning problem expands a greater number of nodes. That is to say, for problems it is uncertain about, it undertakes a greater degree of exploration. Notably, normalized log-likelihood based rewards penalize a greater depth, and hence the degree of exploration is substantially reduced when using it as process reward. We attribute the rather poor test time generalization of the log-likelihood process rewards-based models to this particular feature.

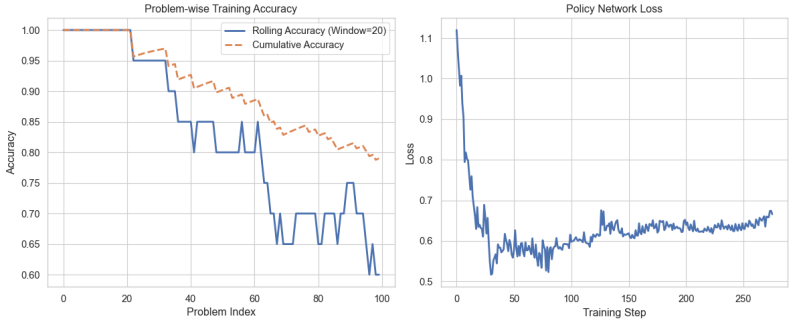
In Figs 4a, 4b, 4c, we can observe that the reasoning models 'try harder' for problems it is uncertain about, adaptively using a greater amount of computation for such problems.



a Training Policy Network with Self-critic Reward

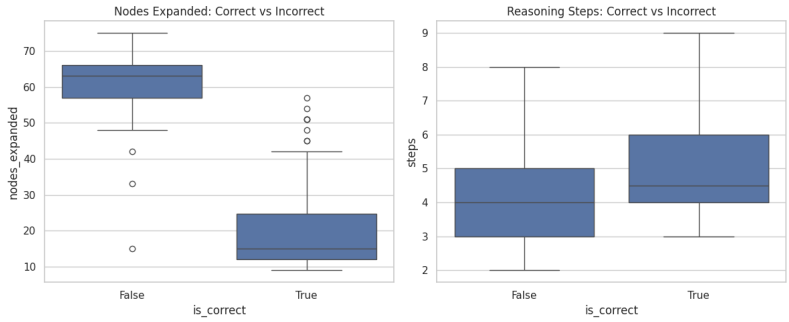


b Training Policy Network with Log-likelihood Process Reward

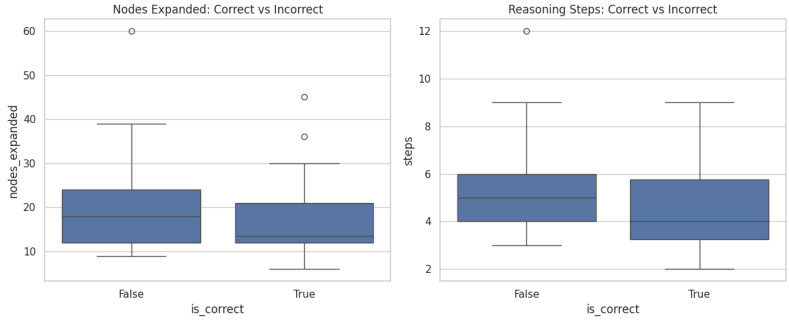


c Training Policy Network without Process Reward

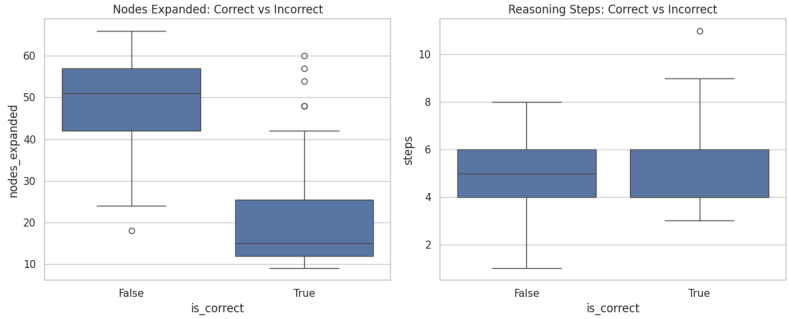
Figure 2: Training accuracy and loss visualization of Policy Networks



a Inference analysis using self-critic reward

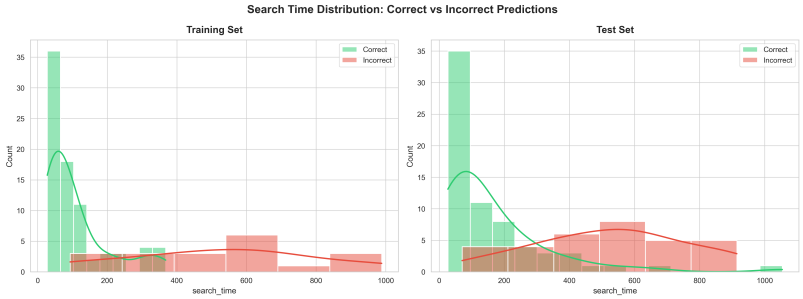


b Inference analysis using log-likelihood reward

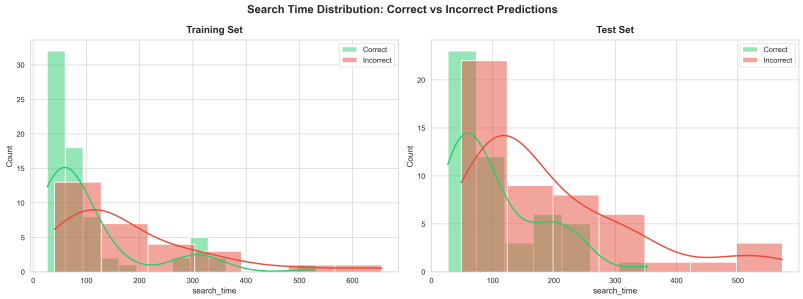


c Inference analysis without process reward

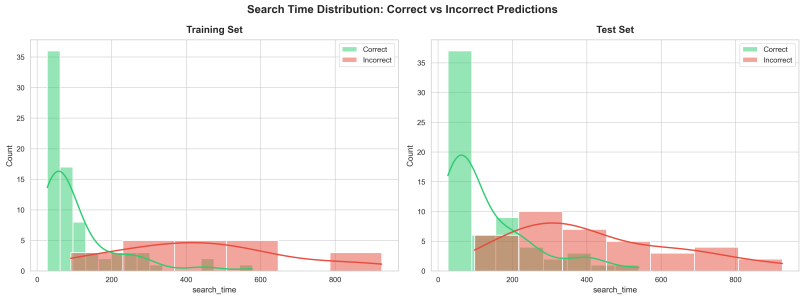
Figure 3: Inference Performance Visualization vs. tree depath



a Search-time distribution when using self-critic-based process rewards



b Search-time distribution when using log-likelihood-based process rewards



c Search-time distribution when not using process rewards

Figure 4: Search-time distribution